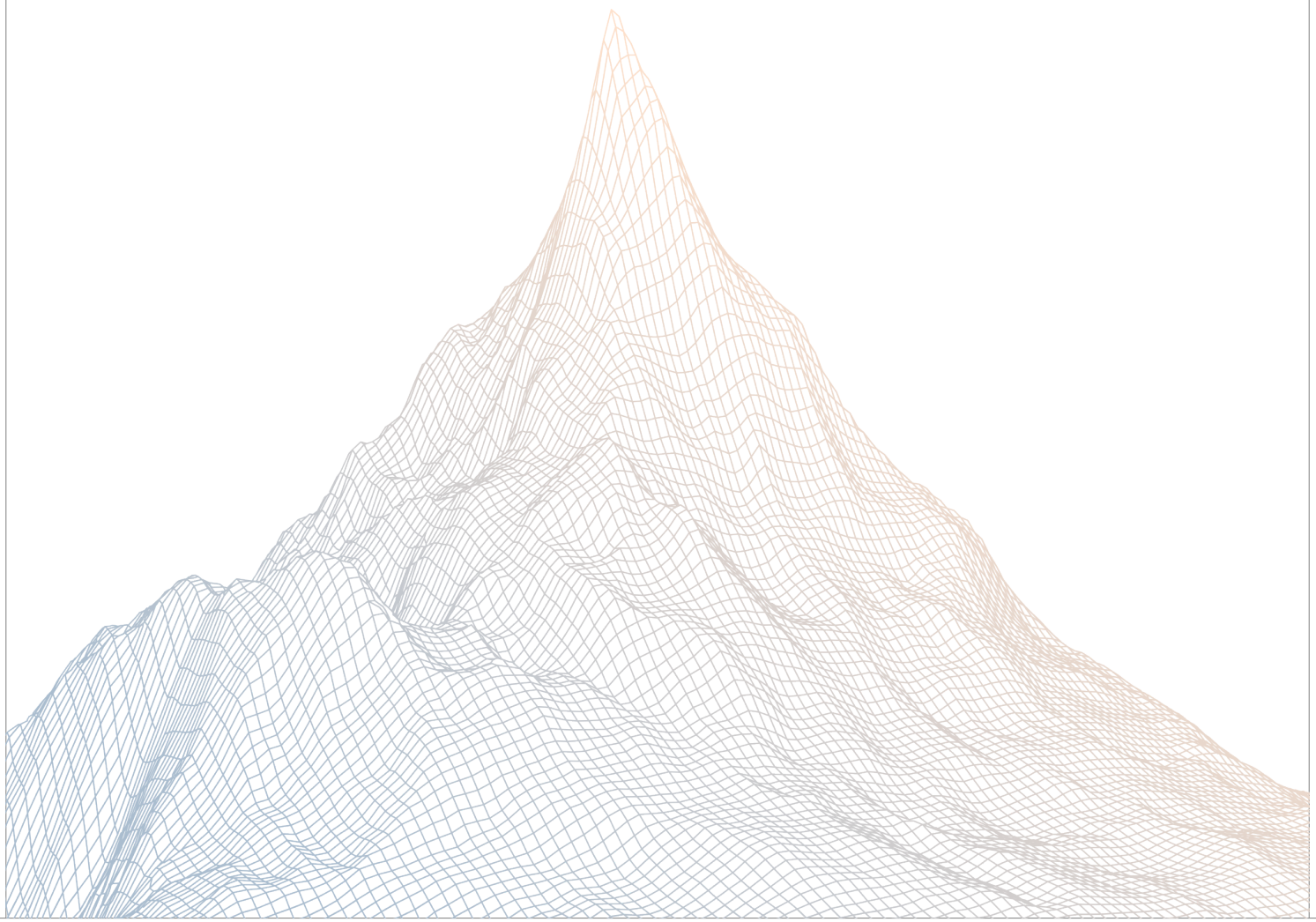
Zenith

Interfold

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: June 29th to July 2nd, 2026

AUDITED BY: ether_sky
said

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Interfold

The Interfold enables applications that coordinate computation across independent parties without exposing their private inputs. Instead of pooling sensitive data or delegating execution authority to a centralized operator, the protocol executes encrypted workflows across a distributed network of ciphernodes.

2.2 Scope

The engagement involved a review of the following targets:

Target	interfold
Repository	https://github.com/theinterfold/interfold
Commit Hash	`0e8e728a7df74cbdab58a159e6d0e0d272321a6a`
Files	`InterfoldTicketToken.sol` `InterfoldToken.sol`

Target	Mitigation Review
Repository	https://github.com/theinterfold/interfold
Commit Hash	`09b20663596232626 44d9619d1b2613220b2e320`
Files	`InterfoldTicketToken.sol` `InterfoldToken.sol`

2.3 Audit Timeline

June 29, 2026	Audit start
July 2, 2026	Audit end
July 2, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	5
Informational	7
Total Issues	**12**

3

Findings Summary

ID	Description	Status
L-1	Stale active lock slots can freeze CCA claims	Resolved
L-2	Multiple queued CCA buckets can assign claims to the wrong lock policy	Resolved
L-3	Token transfers should only be allowed when both the sender and the recipient are whitelisted	Acknowledged
L-4	Pre-TGE relink can revoke accrued Absolute-policy transferability	Resolved
L-5	Absolute policies can be created already fully matured	Resolved
I-1	renounceOwnership is disabled but renounceRole is not	Resolved
I-2	Registry rotation can strand or reroute InterfoldTicketToken payout obligations	Resolved
I-3	TGE-anchored policies are validated against the earliest possible TGE, so a delayed TGE lets the sunset truncate vesting	Acknowledged
I-4	The new registry should be different from the old one	Resolved
I-5	Disabled permit should be removed	Resolved
I-6	payout() should use the existing reentrancy guard	Resolved
I-7	Claim-lock exemption can coexist with stale queued claim buckets	Resolved

4

Findings

4.1 Low Risk

A total of 5 low risk findings were identified.

[L-1] Stale active lock slots can freeze CCA claims

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldToken.sol#193
- InterfoldToken.sol#531-555
- InterfoldToken.sol#632-640
- InterfoldToken.sol#840-871

Description:

Each account can only hold eight active lock entries. The cap counts storage entries, not currently locked value. Fully vested lock entries are never removed from `locks[account]`.

```
uint256 public constant MAX_LOCKS_PER_ACCOUNT = 8;

function lockedBalanceAt(
    address account,
    uint64 timestamp
) public view returns (uint256 lockedBalance) {
    Lock[] storage accountLocks = locks[account];
    for (uint256 i = 0; i < accountLocks.length; i++) {
        Lock storage accountLock = accountLocks[i];
        ...
        lockedBalance += _lockedAmount(lockPolicies[policyId], amount,
    timestamp);
    }
}
```

When a claim-source transfer arrives, `_update()` calls `_claim()` after the ERC20 transfer:

```
if (!isBurn && value ≠ 0 && from == CLAIM_SOURCE && !claimLockExempt[to]) {
    _claim(to, value);
}
```

If the recipient already has eight old active lock entries, `_claim()` can revert while trying to add `PENDING_LOCK_POLICY_ID` or another queued policy:

```
if (isActive && len >= MAX_LOCKS_PER_ACCOUNT) revert TooManyLocks();
entries.push(Lock(policyId, amount));
```

The whole CCA transfer reverts. This can block legitimate claims even when the existing lock entries have fully vested and no longer restrict transfers.

Recommendations:

Prune fully matured active locks before enforcing the active-lock cap.

Interfold: Resolved with `@a55e47fd39 ...`

Zenith: Verified.

[L-2] Multiple queued CCA buckets can assign claims to the wrong lock policy

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldToken.sol#452-477
- InterfoldToken.sol#716-741
- InterfoldToken.sol#753-786
- InterfoldToken.sol#793-838

Description:

`linkClaim()` can queue more than one policy bucket for the same account. The code comments say this case is undefined, but the contract does not reject it. When a claim arrives, `_claim()` consumes the first queued entry in storage. There is no claim id, bucket id, or policy selector in the claim transfer path. Consider this scenario:

1. The lock importer calls `linkClaim(Alice, 100, PolicyA)` before Alice receives the CCA tokens.
2. Alice has no active pending claim yet, so `_linkClaim()` stores the amount in `queuedLocks[Alice]` under `PolicyA`.
3. The importer later calls `linkClaim(Alice, 100, PolicyB)`.
4. The contract allows this too, so Alice now has two queued buckets:

```
queuedLocks[Alice][0] = PolicyA, 100
queuedLocks[Alice][1] = PolicyB, 100
```

5. Alice later receives a CCA transfer from `CLAIM_SOURCE`.
6. `_update()` sees `from == CLAIM_SOURCE` and calls `_claim(Alice, amount)`.
7. `_claim()` does not know whether the transfer belongs to `PolicyA` or `PolicyB`.
8. `_claim()` passes `bytes32(0)` into `_consumeLock()`, which means consume the first queued entry.

9. If this real CCA transfer was meant for `PolicyB`, the contract still consumes `PolicyA` because it is first in storage.

```
function _claim(address account, uint256 amount) private {
    uint256 remaining = amount;
    while (remaining != 0) {
        (uint256 consumed, bytes32 policyId) = _consumeLock(
            account,
            queuedLocks[account],
            bytes32(0),
            remaining,
            false
        );

        if (consumed == 0) {
            policyId = PENDING_LOCK_POLICY_ID;
            consumed = remaining;
        }

        _addOrIncrementLock(account, locks[account], policyId, consumed,
    true);
        remaining -= consumed;
    }
}
```

The mismatch is:

```
Expected: claim #1 -> PolicyB
Actual:   #1 -> PolicyA
```

If `PolicyA` unlocks over 6 months and `PolicyB` unlocks over 4 years, the same CCA tokens can be assigned to the wrong vesting schedule. This can unlock tokens too early or keep them locked longer than intended.

Recommendations:

Reject a second queued CCA policy for the same account when it differs from the existing queued policy, Or add a claim or bucket id to the CCA claim path and consume the matching queued bucket only.

Interfold: Resolved with PR-1638.

Zenith: Verified.

[L-3] Token transfers should only be allowed when both the sender and the recipient are whitelisted

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- InterfoldToken.sol

Description:

`Tokens` should only be transferable when both the `sender` and the `recipient` are `whitelisted`. However, the current implementation allows transfers as long as either the `sender` or the `recipient` is `whitelisted`.

```
function _isTransferRestricted(
    address from,
    address to
) internal view returns (bool) {
    if (tgeTimestamp ≠ 0) return false;
    if (from == address(0) || to == address(0)) return false;

    address registry = address(BONDING_REGISTRY);
    bool isBonding = from == registry || to == registry;
    bool isCcaDistribution = from == CLAIM_SOURCE;
@->    bool isWhitelisted = transferWhitelist[from] ||
    transferWhitelist[to];
    return !isBonding && !isCcaDistribution && !isWhitelisted;
}
```

As a result, transfers between two `non-whitelisted` users are possible by using a `whitelisted account` as an intermediary.

Recommendations:

```
function _isTransferRestricted(
    address from,
    address to
```

```
) internal view returns (bool) {
    if (tgeTimestamp ≠ 0) return false;
    if (from == address(0) || to == address(0)) return false;

    address registry = address(BONDING_REGISTRY);
    bool isBonding = from == registry || to == registry;
    bool isCcaDistribution = from == CLAIM_SOURCE;
    bool isWhitelisted = transferWhitelist[from] || transferWhitelist[to];
    bool isWhitelisted = transferWhitelist[from] && transferWhitelist[to];

    return !isBonding && !isCcaDistribution && !isWhitelisted;
}
```

Interfold: Acknowledged. The whitelist is strictly limited to trusted non-forwarding operational addresses.

[L-4] Pre-TGE relink can revoke accrued Absolute-policy transferability

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- InterfoldToken.sol:486-515
- InterfoldToken.sol:531-572
- InterfoldToken.sol:973-996

Description:

`relinkActiveLock()` moves raw active lock principal from one policy to another. It does not preserve the amount that had already vested under the old policy.

```
function relinkActiveLock(
    address account,
    bytes32 fromPolicyId,
    bytes32 toPolicyId,
    uint256 amount
) external onlyRole(LOCK_MANAGER_ROLE) {
    if (tgeTimestamp ≠ 0) revert AlreadyLive();
    _validateRelinkParams(account, fromPolicyId, toPolicyId, amount);
    if (_activeLockAmount(account, fromPolicyId) < amount) {
        revert RelinkAmountExceeded();
    }

    (uint256 consumed, ) = _consumeLock(
        account,
        locks[account],
        fromPolicyId,
        amount,
        true
    );

    _addOrIncrementLock(account, locks[account], toPolicyId, consumed,
    true);
}
```

For Absolute policies, vesting can accrue before TGE. A user can have transferable tokens under the current policy, then lose that transferability if a lock manager relinks the same raw principal to a later or stricter Absolute policy before the user transfers.

The affected path is a non-bonding transfer already allowed by the transfer gate, such as a whitelisted transfer.

Recommendations:

When relinking an already-started Absolute policy, preserve the vested portion. Move only the still-locked remainder to the new policy. Another option is to reject relinks from Absolute policies that have already started vesting.

Interfold: Resolved with `@ac5c2bbeaf ...`

Zenith: Verified.

[L-5] Absolute policies can be created already fully matured

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldToken.sol#928-945
- InterfoldToken.sol#973-996

Description:

Policy validation checks that a policy does not run past `NO_MORE_LOCKS`. It does not check whether an Absolute policy already ended before it is created.

```
// ...
uint256 policyEnd = curve.anchor == Anchor.Tge
    ? uint256(CCA_END) + TGE_COOLDOWN + curveEnd
    : curve.start + curveEnd;

if (policyEnd > NO_MORE_LOCKS) {
    revert InvalidPolicy();
}
// ...
```

When `_lockedAmount()` evaluates a fully matured Absolute policy, it returns zero:

```
if (vestDuration == 0 || timestamp >= anchor + vestDuration) {
    return 0;
}
```

A new allocation can look like it was minted under a lock policy while being immediately transferable.

Recommendations:

For Absolute policies meant to lock new allocations, reject `policyEnd ⩽ block.timestamp`. If stale policies are needed for imports, add a separate explicit path and make callers opt in.

Interfold: Resolved with PR-1635.

Zenith: Verified.

4.2 Informational

A total of 7 informational findings were identified.

[I-1] renounceOwnership is disabled but renounceRole is not

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldToken.sol

Description:

The contract deliberately disables `renounceOwnership` to avoid bricking governance. But it inherits `AccessControl`'s public `renounceRole`, so the `owner` can still renounce `DEFAULT_ADMIN_ROLE` (or other roles) directly.

Recommendations:

If the intent is that the `owner` can never accidentally strip its own admin powers, override `renounceRole` to revert for the owner

Interfold: Resolved with `@6afee85440 ...`

Zenith: Verified.

[I-2] Registry rotation can strand or reroute `InterfoldTicketToken` payout obligations

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldTicketToken.sol#151-154
- InterfoldTicketToken.sol#187-193
- InterfoldTicketToken.sol#198-202
- InterfoldTicketToken.sol#207-230
- InterfoldTicketToken.sol#318-335

Description:

All privileged `InterfoldTicketToken` operations are gated by the current `registry` address:

```
modifier onlyRegistry() {
    if (msg.sender ≠ registry) revert NotRegistry();
    _;
}
```

Before the registry is locked, the owner can replace it instantly:

```
function setRegistry(address newRegistry) external onlyOwner {
    if (registryLocked) revert RegistryAlreadyLocked();
    if (newRegistry == address(0)) revert ZeroAddress();
    address old = registry;
    registry = newRegistry;
    emit RegistryChanged(old, newRegistry);
}
```

Ticket exits and slashes can create later payout obligations by calling `burnTickets()`, which increases the token-level `payableBalance`:

```
function burnTickets(
    address operator,
```

```
    uint256 amount
) external onlyRegistry {
    payableBalance += amount;
    _burn(operator, amount);
}
```

If the registry is changed after obligations are created, the old registry can no longer call `payout()`. Pending exits can revert with `NotRegistry`. The new registry also controls the same global `payableBalance`, so a bad migration can move funds that back obligations created by the old registry.

Recommendations:

Call `lockRegistry()` immediately after deployment wiring. For future registry changes, pause new ticket exits and slashes, migrate or settle pending obligations, then activate the new registry.

Interfold: Resolved with PR-1636.

Zenith: Verified.

[I-3] TGE-anchored policies are validated against the earliest possible TGE, so a delayed TGE lets the sunset truncate vesting

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- InterfoldToken.sol

Description:

When a `lock policy` is created, `_validatePolicyMaturity` checks that a `TGE-anchored curve` finishes before `NO_MORE_LOCKS`. But it computes the `policy`'s `end` using the `_earliest possible_ TGE` (`CCA_END + TGE_COOLDOWN`), not the actual `TGE` - which isn't known yet. The actual `tge()` can fire later. If it does, the real `curve end` shifts past `NO_MORE_LOCKS`, and the sunset forcibly unlocks everything at `NO_MORE_LOCKS` - cutting the intended vesting short.

Recommendations:

This could be acknowledged.

Interfold: Acknowledged.Intent here is to set a large buffer weeks/months, between the estimated end of the last lock NO_MORE_LOCKS.

[I-4] The new registry should be different from the old one

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldTicketToken.sol

Description:

When switching to a new `registry`, it should be different from the old one.

Recommendations:

```
function setRegistry(address newRegistry) external onlyOwner {
    if (registryLocked) revert RegistryAlreadyLocked();
    if (newRegistry == address(0)) revert ZeroAddress();
    if (newRegistry == registry) revert SameRegistry();

    address old = registry;
    registry = newRegistry;
    emit RegistryChanged(old, newRegistry);
}

function requestRegistryChange(address newRegistry) external onlyOwner {
    if (!registryLocked) revert RegistryNotLocked();
    if (newRegistry == address(0)) revert ZeroAddress();
    if (newRegistry == registry) revert SameRegistry();
    pendingRegistry = newRegistry;
    uint64 activatesAt = uint64(block.timestamp) + REGISTRY_CHANGE_DELAY;
    pendingRegistryActivationTime = activatesAt;
    emit RegistryChangeRequested(newRegistry, activatesAt);
}
```

Interfold: Resolved with @473557d5fe ...

Zenith: Verified.

[I-5] Disabled permit should be removed

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldTicketToken.sol:16-18
- InterfoldTicketToken.sol:51-57
- InterfoldTicketToken.sol:170-172
- InterfoldTicketToken.sol:359-379
- InterfoldTicketToken.sol:433-439

Description:

`InterfoldTicketToken` is non-transferable and approvals are disabled. `approve()` reverts:

```
function approve(address, uint256) public pure override returns (bool) {
    revert TransferNotAllowed();
}
```

`permit()` is also disabled:

```
function permit(
    address,
    address,
    uint256,
    uint256,
    uint8,
    bytes32,
    bytes32
) public pure override {
    revert PermitDisabled();
}
```

Despite that, the contract still imports and inherits `ERC20Permit`, initializes it in the constructor, and exposes permit-related ABI surface:

```
contract InterfoldTicketToken is
    ERC20,
    ERC20Permit,
    ERC20Votes,
    Ownable2Step,
    ERC20Wrapper,
    ReentrancyGuard
```

The reachable impact is unnecessary surface and integration confusion. `permit()` always reverts and transfers between users are blocked, but wallets, indexers, and protocol adapters can still detect a permit-shaped function and assume gasless approvals are supported.

Recommendations:

Remove `ERC20Permit` from the ticket token.

Interfold: Resolved with PR-1631.

Zenith: Verified.

[I-6] `payout()` should use the existing reentrancy guard

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldTicketToken.sol#330-335

Description:

`InterfoldTicketToken` inherits `ReentrancyGuard`, and its other underlying-moving wrapper functions use `nonReentrant`. `payout()` also moves the underlying token, but it does not use the guard:

```
function payout(address to, uint256 amount) external onlyRegistry {
    require(amount <= payableBalance, "Exceeds payable balance");
    payableBalance -= amount;
    SafeERC20.safeTransfer(IERC20(address(underlying())), to, amount);
    emit Payout(to, amount);
}
```

The issue is consistency and defense-in-depth. If the underlying token has callback behavior, or if a future registry path calls `payout()` without its own reentrancy guard, this function becomes the external-call boundary without local protection.

Recommendations:

Add `nonReentrant` to ‘payout().

Interfold: Resolved with PR-1632.

Zenith: Verified.

[I-7] Claim-lock exemption can coexist with stale queued claim buckets

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- InterfoldToken.sol#224-235
- InterfoldToken.sol#413-419
- InterfoldToken.sol#632-640
- InterfoldToken.sol#753-786

Description:

`claimLockExempt[account]` is intended to make claim-source transfers arrive without automatic claim locks.

The edge case is that `InterfoldToken` can also hold queued claim buckets for the same account. A queued bucket means future claim-source tokens should be classified under a specific policy:

```
/// @notice Addresses exempt from automatic claim-source lock creation.
mapping(address account ⇒ bool exempt) public claimLockExempt;

/// @notice Policy buckets for links that arrived before enough claim
///         balance existed to classify them.
mapping(address account ⇒ Lock[] entries) public queuedLocks;
```

`setClaimLockExempt()` does not check whether the account already has queued buckets:

```
function setClaimLockExempt(
    address account,
    bool exempt
) external onlyRole(LOCK_MANAGER_ROLE) {
    if (account == address(0)) revert ZeroAddress();
    claimLockExempt[account] = exempt;
    emit ClaimLockExemptUpdated(account, exempt);
}
```

If a claim-source transfer arrives while the account is exempt, `_claim()` is skipped entirely:

```
if (
    !isBurn &&
    value ≠ 0 &&
    from = CLAIM_SOURCE &&
    !claimLockExempt[to]
) {
    _claim(to, value);
}
```

That means existing queued buckets are not consumed. The transferred tokens arrive unlocked, which is expected for an exempt account, but the old queued bucket remains:

```
claimLockExempt[Alice] = true
queuedLocks[Alice][0] = PolicyA, 100

CLAIM_SOURCE transfers 100 FOLD to Alice

Result:
Alice receives 100 unlocked FOLD
queuedLocks[Alice][0] is still PolicyA, 100
```

If the exemption is later disabled, the stale queued bucket can classify a later unrelated claim under `PolicyA`.

Recommendations:

Prevent conflicting states. `setClaimLockExempt(account, true)` should either revert when `queuedLocks[account].length ≠ 0` or explicitly clear/migrate those queued buckets. Also consider making `linkClaim()` reject new queued buckets while `claimLockExempt[account] = true`.

Interfold: Resolved with PR-1633.

Zenith: Verified.